

Contents

Oracle Drawdown: A Detection Engineering Analysis of RSA Signature Forgery from a Raw Signing Oracle, and Why the Key Never Has to Leave	1
Scope and provenance	1
1. The defender question nobody is answering	2
2. The research	2
3. The synthesis: oracle drawdown	4
4. Detection, ranked by durability	5
5. What this makes undetectable	6
6. What actually found it	7
7. Credit	7
8. Limitations	7
9. What would falsify this	8
10. What we are not claiming	8
11. AI disclosure	8
References	9

Oracle Drawdown: A Detection Engineering Analysis of RSA Signature Forgery from a Raw Signing Oracle, and Why the Key Never Has to Leave

Cristian Ruvalcaba and the Saluca Agentic AI Research Team, Saluca LLC

Version 1.0 - 26 September 2026 - current to 26 September 2026

Sixth in the consolidated detection engineering series, and the first built on cryptanalytic research rather than on an intrusion. It extends the argument of *Detection Without Indicators: Agent-Originated Intrusion* (concept DOI 10.5281/zenodo.21770780), that where an attack leaves no artifact at the point of use, detection has to move to accounting, to a case where the artifact the attacker produces is not merely unindicated but mathematically perfect.

Scope and provenance

Built from one primary source: Laura Shea, Miro Haller, Adam Suhl, Nadia Heninger (University of California San Diego) and Emmanuel Thomé (Inria Nancy), *Forging 1024-bit RSA signatures in nearly SNFS time*, IACR ePrint 2026/2131, dated 20 September 2026. The full paper was read. Every number below attributed to “the paper” or “the authors” is theirs. Secondary coverage was read for context only and carries no technical claim here.

We had no access to the authors’ code, their HSMs, or any vendor. Nothing was reproduced, and nothing in this paper or its companion package generates oracle queries or performs any stage of the attack. The companion’s tests provision a software token (SoftHSM2) to check that the audit tool reads key policy correctly and that restricting a key’s mechanisms actually refuses a raw operation; no real HSM and no HSM audit telemetry were available, and **no detection in the companion package has been run against telemetry from a real signer.**

ATT&CK mappings in the companion are our inference. The paper supplies none, and no ATT&CK technique describes “cause a signer to act as an oracle” exactly.

The caveat that governs the whole paper. This is research, not an incident. There is no victim, no campaign, and no report of the technique being used against anyone. The authors state that they “do not believe that the attack we implement should be cause for immediate alarm among practitioners managing cryptographic inventory in industry”, describing it as “practical in an academic sense rather than in the script-kiddie sense.” Every claim below is about what a defender could see if it were used, not about something that has happened.

1. The defender question nobody is answering

The coverage of this paper asks whether RSA is broken. It is not, and the authors say so. The question a defender needs answered is different: **if this were run against my signer, what would I have seen, and when?**

The honest answer is short and uncomfortable. The attack has three phases. Two of them never touch the defender’s estate. The third produces nothing a verifier can distinguish from legitimate output. The only phase that crosses the defender’s infrastructure is a period of heavy but otherwise ordinary-looking signing in the middle, and against one important class of deployment even that is, in the authors’ words, indistinguishable from legitimate use.

Hardware security modules exist to keep a private key inside a boundary. Nearly every control built around them, including tamper resistance, non-extractability, key ceremonies and FIPS validation, is aimed at the key. This paper demonstrates that the key does not have to leave. What leaves is the capability the key confers, and no control aimed at the key can see that happen.

2. The research

Facts in this section are the authors’.

The algorithm. Joux, Naccache and Thomé described it in 2007. The authors call it the e -th root NFS. It is a variant of the number field sieve that computes RSA e -th roots, and so forges signatures, in time close to the special number field sieve, $L_N(1/3, 1.577)$, rather than the general number field sieve, $L_N(1/3, 1.923)$, that is used to size RSA keys. It had never been run at a meaningful key size.

The threat model. A “lunchtime attack”: “an attacker who obtains temporary access to an RSA signature/decryption oracle for a given key, and then is challenged to generate a signature forgery or ciphertext decryption after losing access.” The oracle must be **raw**: it computes $x^d \bmod N$ on an attacker-chosen x , with no padding. The public key must be known far enough in advance to precompute.

The three phases, at 1024 bits:

Phase	Needs	Cost	Runs on
Precomputation	public key only	~1,200 core-years	attacker’s hardware

Phase	Needs	Cost	Runs on
Queries	the raw oracle	4,067,419,331 queries	the defender’s signer
Forgery, per message	stored results only	~180 core-years	attacker’s hardware

Total 1,380 core-years over five calendar months. The paper cites current estimates of 500,000 to 1,000,000 core-years to factor a 1024-bit modulus.

The queries. Rational primes from 2 to 2^{36} , plus values of the form $a - b \cdot m$ for relations found in precomputation (2,874,398,515 rational, 144,973,282 algebraic and 1,048,047,534 extension queries). The query set is fixed by the precomputation; it is not adaptive.

The oracle. A Thales Luna K6 HSM with FIPS-approved mode disabled to permit raw RSA (CKM_RSA_X_509 in PKCS#11), averaging about 6,500 signatures per second on 40 threads, and a Thales Luna S750 operated by a third party on another continent, reached over the internet at roughly 2,500 signatures per second. The authors: “Even currently used HSMs can be in a configuration state that allows us to run our attack. Furthermore, the Luna S750 HSM test shows that it is possible for an adversary to run these queries over the internet.” On the K6 the query phase would have taken about 175 hours uninterrupted.

Larger keys. Extrapolated, not demonstrated: 2^{65} work for 1024-bit (against the usual 80-bit rating), 2^{90} work and 2^{43} queries for 2048-bit, 2^{119} work and 2^{57} queries for 4096-bit. The reduction against factoring-based estimates is 15 to 30 bits; “even 4096-bit RSA does not appear to meet a 128-bit security level in this attack model.” The figures are presented as an upper bound; the authors estimate 2 to 9 GPU-years for the 1024-bit forgery using recent GPU sieving work.

What is not exposed. “The vast majority of RSA use that we are aware of in the modern era is for digital signatures using PKCS#1v1.5 or RSA-PSS padding. [...] an RSA oracle that applies such padding does not seem to provide much of an asymptotic speed improvement over factoring for signature forgery.” TLS, OAuth RS256, DNSSEC and DKIM, all surveyed in the paper, use padded signatures and provide no obvious raw oracle. The results “do not carry over to other popular signature algorithms such as ECDSA or Ed25519.”

What is exposed. Where raw oracles arise in practice, per the paper:

- **RSA blind signatures**, RFC 9474, which “provide exactly such oracle access to the attacker, allowing it to make (blinded) oracle queries that are indistinguishable from the valid blinded signature queries.” Named deployments: the publicly verifiable Privacy Pass token type, implemented by Apple Private Access Tokens and used by Fastly, Persona and Cloudflare; Apple iCloud Private Relay; GNU Taler.
- **HSMs with raw RSA enabled**, typically because the application applies a padding the HSM does not implement: ISO 9796-2 (e-passports, digital tachographs), ANSI X9.31, RSA-FDH.
- **Platform keystores**: Android KeyMint permits unpadded RSA decryption.
- **Padding oracles as a stepping stone**: a Bleichenbacher oracle can manufacture raw queries at much higher cost, converting a vulnerability that ends with the patch into a capability that does not.

The authors’ mitigations for blind RSA: “rotating keys frequently as a short-term mitigation, transitioning to larger key sizes in the medium term, and adding ZK proofs or transitioning to post-

quantum schemes in the long term.”

3. The synthesis: oracle drawdown

This section is ours.

The key is not the asset. The capability is. Key custody answers the question “can the secret leave?” This attack answers a different one: “can the capability leave without the secret?” Every private-key operation a signer performs for someone else is a small, irrevocable transfer of what the key is for. For padded operations on honestly formed messages the transfer is worthless beyond the one signature. For raw operations on chosen inputs, enough of them accumulate into the whole capability. We call this **oracle drawdown**: a raw signer’s integrity is a finite balance, each chosen-input operation draws on it, and the attack is the point at which the balance is exhausted.

The unusual property: the attacker’s price is published. In most intrusions a defender does not know how much activity an attacker needs before succeeding. Here the paper states it: about 2^{32} raw queries against a 1024-bit key, and an extrapolated 2^{43} against 2048-bit. That is a minimum consumption the attacker cannot avoid, fixed before the first query, and it turns detection into arithmetic. For a key performing q chosen-input operations per day, the attack’s query price is reached after $2^{32} / q$ days at 1024 bits and $2^{43} / q$ days at 2048 bits. A 1024-bit key serving a billion operations a day reaches the 1024-bit price in about four days; a 2048-bit key at the same rate needs about 24 years to reach the 2048-bit price. The same formula tells an operator whether their rotation interval is doing any work.

Budgets beat rates. The attack does not need speed. The researchers’ first batch of rational queries took over a month on their local HSM because of debugging pauses, and the remote HSM ran at under half the local one’s rate. Neither slowed the attack in any way that mattered. A rule keyed on bursts is keyed on the attacker’s convenience. A rule keyed on the running total per key since creation is keyed on the attacker’s necessity. The drawdown framing makes the correct rule obvious: **count cumulatively, per key, against a budget set as a fraction of the published price**, and rotate before the budget is spent.

The precomputation window. Precomputation depends on the public key and has to finish before the queries begin. A key therefore has to be public for at least the precomputation time plus the query time before it can be drawn down. At 2048 bits, 2^{90} work makes that window irrelevant today. At 1024 bits it is not: 1,200 core-years spread over 100,000 cores is under five days. For any 1024-bit raw signer still in service, publishing the next key late and retiring each key quickly is a real control, not a formality.

Ledger reconciliation is the only post-hoc detection. A forged signature is valid. The only thing that distinguishes it from a real one is that the real one was recorded when it was made. If the signing path writes a ledger, then any signature observed verifying under the key and absent from the ledger is a forgery, regardless of how it was produced. For token issuers the ledger collapses to a count: per key, distinct redemptions can never legitimately exceed issuances.

The extension we are least sure of: delegated signing to autonomous agents. The paper’s oracle is an HSM behind an API. Autonomous agents are increasingly given exactly that: a tool that signs, with credentials, reachable at machine speed and without fatigue. An agent does not need to be malicious to be an oracle client; it needs only to be steered, and earlier papers in this

series documented agents taking actions their operators had not intended. The drawdown framing applies unchanged: an agent’s signing tool is a raw oracle if it passes caller-chosen bytes to a raw mechanism, and its safety is bounded by the same budget arithmetic. We have no incident to cite. This is a prediction about where the next raw oracle will be exposed, not an observation.

3.1 Where this is weaker

- **Budgets need per-operation counts, and many HSMs do not produce them.** Audit logs commonly cover administration, authentication and policy, not each signing operation. Where the count is not available at the HSM it must be taken at the application or gateway, which an attacker with direct HSM access bypasses. The paper’s own threat model includes a partially compromised server in front of the HSM, which is exactly that attacker.
- **At 2048 bits the budget is rarely the binding constraint.** A key has to perform 2^{43} chosen-input operations before the budget matters, and the 2^{90} precomputation is the real barrier today. The drawdown framing is most useful for 1024-bit keys still in service, for high-volume blind signers, and as a planning tool for when the compute cost falls.
- **Ledger reconciliation needs an observation feed.** Where you never see your signatures in use, you cannot reconcile them. Certificate Transparency provides this for public certificates; most internal signing does not have an equivalent.
- **The query set is non-adaptive, which helps only against a careless attacker.** Because the queries are fixed in advance and small, an unblinded attack is trivially recognisable. But any attacker with the public key can blind every query at negligible cost, so the recognisability is not a property a defender can rely on.
- **The agent extension is speculative.** It is included because the series has repeatedly found that agents are the first consumers of new capability surfaces, not because anything has happened.

4. Detection, ranked by durability

Ranked by whether the detection survives an attacker who has read this paper, not by ease of deployment. “Survives” means it still works when the attacker blinds their queries, spreads them over time, and uses many clients.

#	Detection	Survives a careful attacker	Needs tuning	Decays	Companion
1	Signing-ledger reconciliation; per-key redemptions never exceed issuances	yes	ledger completeness	no	SPL 1, 3; KQL 1, 3; <code>rsa_oracle_audit.py reconcile</code>
2	Cumulative per-key private-key operation budget against the published price	yes	per-key forecast	as the query price falls	Sigma correlation; SPL 2; KQL 2

#	Detection	Survives a careful attacker	Needs tuning	Decays	Companion
3	Key policy permits CKM_RSA_X_509, or has no mechanism restriction at all	yes, but only finds the precondition	allowlist of justified raw keys	no	rsa_oracle_audit.py pkcs11; Sigma 1
4	HSM policy change disabling FIPS-approved mode or permitting raw RSA	yes, but only at the moment of change	vendor vocabulary	no	Sigma 3
5	Padding-structure gate before any software-padded raw call	yes (a control)	the padding scheme in use	no	rsa_oracle_audit.py shape --expect-padding
6	Raw RSA in source code and command lines	inventory only	exclusions	no	Semgrep (11 rules); Sigma 4
7	Raw-operation inputs far shorter than the modulus	no (blinding defeats it)	none	immediately, against anyone careful	SPL 4; KQL 4; rsa_oracle_audit.py shape

Deploy first: 1, 2 and 3. Detection 1 is the only one that works after the attacker has gone and the only one blinding cannot touch. Detection 2 is the only one that sees the attack itself. Detection 3 is the cheapest, and in the companion’s test against a software token, restricting a key’s allowed mechanisms to PSS caused the token to refuse the raw operation outright. That turns a detection into a prevention for every key that does not need raw RSA.

Detection 7 is included as a tripwire and labelled as one. It catches the attack exactly as the researchers ran it. A quiet result means nothing.

5. What this makes undetectable

- **The forged signature.** It is a valid RSA signature on the attacker’s chosen message. No verifier-side rule, anomaly model or signature inspection can distinguish it. Detection at the point of use is not merely difficult; it is structurally unavailable.
- **Two of three phases.** Precomputation needs only the public key; forgery needs only stored results. Neither touches the defender. There is no network, host or HSM telemetry for either.
- **Blind RSA queries, by content.** The paper states that blinded attack queries are indistinguishable from valid ones. For RFC 9474 deployments there is no query-level detection; only volume and reconciliation remain.
- **Queries from a blinding attacker against any raw signer.** Even where the defender can see raw inputs, blinding makes them uniformly random. Only a structural padding check refuses them, and only where the application has a padding structure to check.
- **Queries at the HSM where only the HSM sees them.** If the attacker has direct access to the HSM (the paper’s partially compromised server), any count taken at an application or gateway is bypassed, and many HSMs do not count per operation themselves.

- **The moment of success.** Nothing observable happens when the attacker completes the forgery. The defender’s first opportunity after the query phase is the forged signature’s use, and only a ledger can see that.
-

6. What actually found it

Not a detection, an incident response, or a victim. An academic research project, which the authors credit to a question from Henry Corrigan-Gibbs, implementing a 19-year-old algorithm that the community had not run at scale.

This is the sixth paper in this series in which the thing that surfaced the threat was not the defensive stack of anyone it could affect. Here there is not even an affected party. The authors make the point themselves: “The algorithmic result we use has been known for two decades but the impact on RSA parameter choices does not appear to have been understood in the community.” For a defender, the useful question is how much else in their key inventory is sized against a threat model that a known algorithm quietly invalidated.

7. Credit

This paper exists because Laura Shea, Miro Haller, Adam Suhl, Nadia Heninger and Emmanuel Thomé did the work, published their code, stated their limitations plainly, and wrote a section on concrete deployments that most cryptanalytic papers leave to others. Their threat-model statement, their refusal to overclaim, and their deployment analysis are the reason a defender’s paper can be written a week later. Credit also to Joux, Naccache and Thomé for the 2007 algorithm, and to the parties the authors thank for HSM access, including Matt Green and the team at ZeroRISC.

8. Limitations

- One primary source, a preprint, not yet peer reviewed. The 2048 and 4096-bit figures are the authors’ extrapolations.
 - No HSM and no HSM audit telemetry were available. The companion’s Sigma, SPL and KQL logic has been syntax-checked (Sigma with the official linter) but not run against real signer logs. The audit tool’s PKCS#11 mode was tested only against SoftHSM2, not against a commercial HSM, and vendors differ in how they report `CKA_ALLOWED_MECHANISMS`.
 - The companion uses a neutral log schema because no common schema for HSM operations exists. Every deployment requires a mapping, and a wrong mapping produces a silent rule.
 - The budget arithmetic in section 3 assumes the published query counts. Better implementations may need fewer queries; the authors describe query minimisation as a separate optimisation goal.
 - We did not assess any named deployment. Statements about Privacy Pass, Apple, Cloudflare, Fastly, Persona and GNU Taler are the authors’, reported here, not findings of ours.
-

9. What would falsify this

1. **A comparable attack through a padded oracle.** If a variant achieves similar cost through a PKCS#1 v1.5 or PSS signing oracle, the padding gate (detection 5) and the claim that padded signers are not meaningfully exposed both fail. The authors assess this as not currently the case.
 2. **A large reduction in query count.** If the queries needed for 2048-bit fall by many orders of magnitude, budgets (detection 2) stop separating the attack from normal use for high-volume keys.
 3. **Precomputation that does not depend on the individual key.** If most of the work can be shared across moduli, rotation stops resetting the attacker’s investment and the pre-computation window in section 3 closes.
 4. **Per-operation counting turns out to be standard.** If commercial HSMs in general expose per-key operation counts, our claim that many do not is wrong, and detection 2 is easier to deploy than we say. We would welcome being wrong about this.
 5. **Ledger reconciliation proves too noisy.** If ledger gaps in real signing systems produce false positives faster than they can be investigated, detection 1 becomes a hunt rather than an alert.
-

10. What we are not claiming

- **RSA is not broken.** It is weaker than its key sizes suggest in one access model, and most deployments are not in that model.
 - **Nobody has been attacked.** There is no victim, and nothing here suggests the technique has been used.
 - **No named deployment is at fault.** Blind RSA is a raw oracle by definition; that is what blind signing is. The paper’s own analysis of Apple’s per-device token rate and Persona’s per-call price shows how far practical deployments sit from the 2^{43} queries needed at 2048 bits.
 - **This is not quantum.** It is classical cryptanalysis, and its implication for migration planning is additive to the post-quantum argument, not a substitute for it.
 - **The detections are not complete.** Against blind RSA, the queries are invisible by design, and the companion package says so before it says anything else.
 - **The agent extension in section 3 is not an observation.** It is a prediction, labelled as one.
-

11. AI disclosure

Written with AI assistance. The drawdown framing in section 3, the durability ranking, the falsification conditions and the limitations were drafted with AI assistance and are reviewed by the human author before deposit, who is responsible for every claim. The “what we are not claiming” and “what would falsify this” sections exist because AI-assisted analysis tends to overstate both novelty and coverage.

References

Primary

- Shea, L., Haller, M., Suhl, A., Heninger, N., Thomé, E., *Forging 1024-bit RSA signatures in nearly SNFS time*, IACR Cryptology ePrint Archive 2026/2131, 20 September 2026. <https://eprint.iacr.org/2026/2131> . Code: <https://github.com/ucsd-hacc/NSNFSSSFNFN>

Background

- Joux, A., Naccache, D., Thomé, E., *When e -th roots become easier than factoring*, ASIACRYPT 2007.
- Denis, F., Jacobs, F., Wood, C. A., *RSA Blind Signatures*, RFC 9474, October 2023. <https://www.rfc-editor.org/rfc/rfc9474>
- Moriarty, K. (ed.) et al., *PKCS #1: RSA Cryptography Specifications Version 2.2*, RFC 8017, November 2016. Source of the EMSA-PKCS1-v1_5 structure used by the padding gate.
- OASIS, *PKCS #11 Cryptographic Token Interface*, for CKM_RSA_X_509 and CKA_ALLOWED_MECHANISMS.

This series

- *Detection Without Indicators: Agent-Originated Intrusion*, concept DOI 10.5281/zenodo.21770780.
- *The Quorum Signature*, concept DOI 10.5281/zenodo.22903268.
- *The Coordination Substrate*, concept DOI 10.5281/zenodo.22678061.
- *Asserted Egress*, concept DOI 10.5281/zenodo.22314504.
- *Agentic Intrusion*, concept DOI 10.5281/zenodo.22033405.
- *Borrowed Trust*, concept DOI 10.5281/zenodo.21880001.

Companion detection content

- `saluca-labs/detection-content`, `campaigns/2026-09-rsa-raw-oracle` and `hunt/rsa_oracle_audit.py`, Apache-2.0. <https://github.com/saluca-labs/detection-content>